

s, a, t, r and rx denote string, array, table, range and compiled regular-expression values. Square brackets mark optional parameters; ... accepts additional values.

GLOBALS	
INTERFACE	DESCRIPTION
arg(key, [default])	Read a named command-line argument
assert(condition, [message])	Raise if a condition is false
exec(code)	Execute Tiri source code
forEach(target, callback)	Call a function for each iterator result
getmetatable(value)	Return a value's metatable
ipairs(t)	Iterate over sequential index-value pairs
isthunk(value)	Test whether a value is an unresolved thunk
keys(t)	Iterate over all keys
loadFile(path)	Execute a Tiri source file
msg(...)	Write an API-level log message
next(t, [previous_key])	Return the next table entry
pairs(t)	Iterate over all key-value pairs
print(...)	Write values to standard error
rawequal(a, b)	Compare values without metamethods
rawget(t, key)	Read a table entry without metamethods
rawlen(t)	Return table length without metamethods
rawset(t, key, value)	Write a table entry without metamethods
rawtype(value)	Return the parameterised runtime type
resolve(thunk)	Evaluate a thunk
setmetatable(t, metatable)	Set a table's metatable
tonumber(value, [base])	Convert a value to a number
tostring(value)	Convert a value to a string
type(value)	Return the display type name
values(t)	Iterate over all values

STRINGS	
INTERFACE	DESCRIPTION
string.alloc(size)	Allocate a mutable string buffer
string.byte(s, [start], [stop])	Return byte values from a string
string.cap(s)	Capitalise the first character
string.char(...)	Create a string from byte values
string.count(s, keyword)	Count non-overlapping occurrences
string.decap(s)	Lowercase the first character
string.endsWith(s, suffix)	Test whether a string ends with a suffix
string.escXML(s)	Escape XML content

STRINGS (CONT.)	
INTERFACE	DESCRIPTION
string.find(s, keyword, [start])	Find the first literal occurrence
string.format(format, ...)	Format a string with printf syntax
string.hash(s, [case_sensitive])	Calculate a 32-bit string hash
string.lower(s)	Convert string to lowercase
string.pop(s, [count])	Remove trailing characters
string.rep(s, count, [separator])	Repeat a string
string.replace(s, search, replacement, [limit])	Replace literal occurrences
string.reverse(s)	Reverse the byte order
string.rtrim(s)	Remove trailing whitespace
string.split(s, [separator])	Split a string into an array
string.startsWith(s, prefix)	Test whether a string starts with a prefix
string.sub(s, start, [stop])	Extract a substring
string.toArray(s)	Copy string bytes to an array
string.trim(s)	Remove leading and trailing whitespace
string.unescapeXML(s)	Decode XML entities
string.upper(s)	Convert string to uppercase

ARRAYS	
INTERFACE	DESCRIPTION
a.all(predicate)	Test whether all elements match
a.any(predicate)	Test whether any element matches
a.clear()	Reset the array length to zero
a.clone()	Copy the array
a.concat([separator], [format], [start], [stop])	Concatenate elements into a string
a.copy(source, [dest_index, src_index, count])	Copy values into the array
a.each(callback)	Call a function for each element
a.fill(value, [start, [stop]])	Fill an index span with a value
a.fill(value, range)	Fill a range with a value
a.filter(predicate)	Copy elements that match a predicate
a.find(predicate)	Find the first matching element
a.findIndex(predicate)	Find the first matching element index
a.first()	Return the first element
a.getString([start, length])	Extract a string from a byte array

ARRAYS (CONT.)	
INTERFACE	DESCRIPTION
a.indexOf(value, [start], [stop])	Find a value within an index span
a.indexOf(value, range)	Find a value within a range
a.insert(index, value, ...)	Insert elements at an index
a.last()	Return the last element
a.map(callback, [element_type])	Transform elements into a new array
a.mapSame(callback)	Transform elements preserving their type
array.new(type, size)	Create a typed array of a fixed size
array.of(type, value, ...)	Create a typed array from values
a.pop([count])	Remove and return trailing elements
a.push(value, ...)	Append elements
a.readOnly()	Test whether the array is read-only
a.reduce(initial, reducer)	Fold elements into one value
a.remove(index, [count])	Remove elements at an index
a.resize(new_size)	Change the array length
a.reverse()	Reverse elements in place
a.setString(s, [start])	Write a string to a byte array
a.slice(range)	Extract a range into a new array
a.sort([descending])	Sort elements in place
a.sort(comparator)	Sort elements with a comparator
a.table()	Copy the array to a table
a.type()	Return the runtime array type

TABLES	
INTERFACE	DESCRIPTION
table.clear(t)	Remove all table entries
table.concat(t, [separator, start, stop])	Concatenate sequential elements
table.empty(t)	Test whether a table has no entries
table.insert(t, [pos,] value)	Insert a sequential element
table.kind(t)	Return the table classification
table.move(t, start, stop, target, [dest_table])	Copy a span of elements
table.new(array_size, hash_size)	Create a pre-allocated table
table.remove(t, [pos])	Remove and return a sequential element
table.size(t)	Count all live entries
table.slice(t, range)	Extract a range into a new table
table.sort(t, [comparator])	Sort sequential elements in place
table.sortByKey(t, [comparator])	Iterate in sorted key order
table.toXML(t)	Convert a table to XML

RANGES	
INTERFACE	DESCRIPTION
r.all(predicate)	Test whether all values match
r.any(predicate)	Test whether any value matches
r.each(callback)	Call a function for each value
r.filter(predicate)	Copy matching values to an array
r.find(predicate)	Find the first matching value
r.findIndex(predicate)	Find the first matching ordinal
r.indexOf(value)	Find the ordinal of a value
r.map(transform, [element_type])	Transform values into an array
range.new(start, stop, [inclusive], [step])	Create a range
range(start, stop, [inclusive], [step])	Create a range
r.reduce(initial, reducer)	Fold values into one result
range.slice(object, range)	Extract a range from a collection
r.take(count)	Copy the first values to an array
r.toArray()	Convert a range to an array

MATHEMATICS	
INTERFACE	DESCRIPTION
math.abs(x)	Return the absolute value
math.acos(x)	Calculate the inverse cosine
math.asin(x)	Calculate the inverse sine
math.atan(x)	Calculate the inverse tangent
math.atan2(y, x)	Calculate the two-argument inverse tangent
math.ceil(x)	Round towards positive infinity
math.clamp(value, lower, upper)	Constrain a value to an interval
math.cos(x)	Calculate the cosine
math.cosh(x)	Calculate the hyperbolic cosine
math.deg(x)	Convert radians to degrees
math.exp(x)	Raise e to a power
math.floor(x)	Round towards negative infinity
math.frexp(x)	Split a number into mantissa and exponent
math.ldexp(mantissa, exponent)	Multiply a mantissa by a power of two
math.log(x, [base])	Calculate a logarithm
math.log10(x)	Calculate a base-10 logarithm
math.max(x, ...)	Return the largest argument
math.min(x, ...)	Return the smallest argument
math.modf(x)	Split integer and fractional parts
math.rad(x)	Convert degrees to radians
math.random([low], [high])	Generate a pseudo-random number

MATHEMATICS (CONT.)	
INTERFACE	DESCRIPTION
math.randomSeed(seed)	Seed the random number generator
math.round(x)	Round to the nearest integer
math.sin(x)	Calculate the sine
math.sinh(x)	Calculate the hyperbolic sine
math.sqrt(x)	Calculate the square root
math.tan(x)	Calculate the tangent
math.tanh(x)	Calculate the hyperbolic tangent

REGULAR EXPRESSIONS	
INTERFACE	DESCRIPTION
regex.escape(text)	Escape regular-expression characters
rx.extract(text, [offset], [flags])	Extract groups from the first match
rx.findAll(text, [offset], [flags])	Iterate over all matches
rx.findFirst(text, [offset], [flags])	Find the first match
rx.match(text, [flags])	Match the complete string
regex.new(pattern, [flags])	Compile a regular expression
rx.replace(text, replacement, [flags])	Replace matching text
rx.search(text, [flags])	Return all matches
rx.split(text, [flags])	Split text at match boundaries
rx.test(text, [flags])	Test whether a match exists

I/O	
INTERFACE	DESCRIPTION
file.close()	Close a file handle
io.close([file])	Close a file handle
io.ui.copy(source, dest, [move])	Copy or move files with transfer UI
io.ui.delete(paths)	Delete paths with transfer UI
file.flush()	Flush a file's write buffer
io.flush()	Flush the default output buffer
io.input([source])	Get or set the default input
io.isFolder(path)	Test whether a path ends with a separator
file.lines()	Iterate over lines in a file
io.lines([source])	Iterate over input lines
io.open(path, [mode])	Open a file
io.output([source])	Get or set the default output
io.ui.paste(dest)	Paste clipboard files with transfer UI
file.read([format, ...])	Read data from a file
io.read([format, ...])	Read from the default input
io.readAll(path)	Read an entire file

I/O (CONT.)	
INTERFACE	DESCRIPTION
io.sanitisePath(path)	Remove repeated path separators
io.search(path, options)	Search for matching files
file.seek([whence, [offset]])	Get or set the file position
io.splitPath(path)	Split a path into directory and file
io.tempFile()	Create an in-memory file buffer
io.type(value)	Return a file handle's state
file.write(data, ...)	Write data to a file
io.write(data, ...)	Write to the default output
io.writeAll(path, content)	Write an entire file

STRUCTURES	
INTERFACE	DESCRIPTION
struct.clone(source)	Deep-copy a structure
struct.copy(dest, source)	Deep-copy into a structure
struct.def(struct_name)	Return a structure constructor
struct.new(struct_name, [fields])	Create and initialise a structure
struct.new(struct_name, address)	Reference structure data at an address
inst.ptr([address])	Get or replace the backing address
struct.size(target)	Return a structure's byte size

OBJECTS	
INTERFACE	DESCRIPTION
object.children([class_name])	Return child object IDs
obj.class(object)	Return an object's MetaClass
object.detach()	Convert a strong reference to weak
object.exists()	Test whether an object is alive
obj.find(name, [class_name])	Find an object by name
obj.find(object_id, [class_name])	Find an object by ID
object.free()	Destroy the underlying resource
object.get(field_name, [default])	Read a field by name
object.getKey(key, [default])	Read a key-value entry
object.init()	Initialise an object
obj.new(class_name, [fields])	Create an object
object.set(field_name, value)	Write a field by name
object.setKey(key, value)	Write a key-value entry
object.state()	Return the script state table
object.subscribe(action_name, callback, [custom_ref])	Subscribe to an action or method
object.unsubscribe(action_name)	Remove an action or method subscription

PROCESSING	
INTERFACE	DESCRIPTION
processing.collect([mode], [options])	Run a garbage-collection cycle
processing.delayedCall(function)	Schedule a function for the next cycle
proc.flush()	Clear all monitored signal flags
processing.flush()	Clear the script signal flag
processing.gcStats()	Return garbage-collector statistics
processing.halt(seconds)	Process messages for a fixed duration
processing.new([options])	Create a processing object
proc.signal()	Signal a processing object
processing.signal()	Set the script signal flag
proc.sleep([seconds])	Wait on a processing object
processing.sleep([seconds])	Enter the message-processing loop
processing.startCollector()	Start automatic garbage collection
processing.stopCollector()	Stop automatic garbage collection
subscribeEvent(event_name, callback)	Subscribe to a system event
processing.task()	Return the current process task
unsubscribeEvent(handle)	Remove a system event subscription

ASYNCHRONOUS CALLS	
INTERFACE	DESCRIPTION
async.action(object, name, [callback], [key], [args ...])	Run an action asynchronously
async.cancel(objects)	Cancel pending asynchronous actions
async.pool.clear()	Clear the thread-safe value pool
async.method(object, name, [callback], [key], [args ...])	Run a method asynchronously
async.pending(object)	Count queued and active actions
async.script(script, [callback])	Run a script in another thread
async.wait(objects, [timeout])	Wait for queued work to complete

DEBUGGING	
INTERFACE	DESCRIPTION
debug.fileSources()	Return loaded source files
debug.anno.get(function)	Return a function's annotations
debug.getEnv(function)	Return a function's environment
debug.getHook()	Return the active debug hook
debug.getInfo(target, [what])	Return function or stack information
debug.getLocal(level, index)	Return a local variable
debug.getLocal(function, index)	Return a declared parameter name
debug.getMetatable(object)	Return a metatable without protection

DEBUGGING (CONT.)	
INTERFACE	DESCRIPTION
debug.getRegistry()	Return the registry table
debug.getUpvalue(function, index)	Return a function upvalue
debug.getUserValue(userdata)	Return a userdata value table
debug.anno.list()	Return all annotation entries
debug.locality(name, [level])	Determine where a name is declared
debug.anno.set(function, annotations, [source], [name])	Register function annotations
debug.setEnv(function, t)	Set a function's environment
debug.setHook(hook, mask, [count])	Set a debug hook
debug.setLocal(level, index, value)	Set a local variable
debug.setMetatable(object, t)	Set a metatable without protection
debug.setUpvalue(function, index, value)	Set a function upvalue
debug.setUserValue(userdata, t)	Set a userdata value table
debug.traceback([message, [level]])	Return a stack trace
debug.upvalueID(function, index)	Return an upvalue identifier
debug.upvalueJoin(function1, n1, function2, n2)	Make two upvalues share storage
debug.validate(code, [options])	Validate Tiri source without running it

JIT	
INTERFACE	DESCRIPTION
jit.attach(callback, event)	Register a compiler event callback
jit.flush([target])	Clear compiled code
jit.util.funcBC(function, pc)	Return a bytecode instruction
jit.util.funcInfo(function, [pc])	Return function metadata
jit.util.funcK(function, index)	Return a function constant
jit.util.funcUName(function, index)	Return an upvalue name
jit.util.irCallAddr(index)	Return an IR call address
jit.off([target], [recursive])	Disable JIT compilation
jit.on([target], [recursive])	Enable JIT compilation
jit.opt.start(...flags)	Configure JIT optimisation
jit.status()	Return JIT status and flags
jit.util.traceExitStub([trace_no], exit_no)	Return a trace exit-stub address
jit.util.traceInfo(trace_no)	Return trace metadata
jit.util.traceIR(trace_no, index)	Return trace IR metadata
jit.util.traceK(trace_no, index)	Return a trace constant
jit.util.traceMC(trace_no)	Return trace machine code
jit.util.traceSnap(trace_no, snap_no)	Return a trace snapshot